National College of Ireland

H9IAPA: Intelligent Agents and Process Automation

Automated Clothing Returns & Quality Assurance

An RPA and Agentic AI Pipeline for E-Commerce

Student Name:	[Your Full Name]
Student ID:	[Your Student ID]
Course:	MSc Artificial Intelligence (MSCAIBUS1 / PGDAIBUS_SEP25)
Module:	H9IAPA — Intelligent Agents and Process Automation
Submission:	Thursday, 30 April 2026 at 23:55

AI Disclosure: Report structure, section headings, and code scaffolding were drafted with AI assistance (Claude, Anthropic, 2024). All business process analysis, design decisions, academic arguments, and implementation logic are the student's own original work. No AI tool was used to fabricate or manipulate data, code results, or experimental outcomes.

---

1. Process Identification and Description

The domain selected for this project is **Retail E-Commerce — Clothing**. The process under investigation is the **Customer-Initiated Clothing Returns and Quality Assurance Pipeline**. Online fashion retailers face return rates of 20–30% of total orders (Narvar, 2023), making returns management one of the highest-volume operational processes in the sector. The manual handling of these returns is slow, error-prone, and expensive: a single return typically involves four human touchpoints and takes 3–5 business days to resolve.

The process involves a structured mix of rule-based tasks — verifying purchase dates and policy flags — and cognitively demanding tasks such as visually inspecting garment photographs for defects such as stains, tears, pilling, or missing buttons. This combination makes the process an ideal candidate for hybrid automation: deterministic rules handled by RPA, and visual reasoning handled by an AI agent.

The selected process is relevant to the domain because clothing returns are governed by well-defined business rules (30-day policy window, hygiene item exclusions, final-sale exclusions) that lend themselves directly to automation, while the quality assessment step requires the kind of perception-based reasoning where modern large vision models excel (Russell & Norvig, 2020).

2. Process Modelling (BPMN 2.0)

2.1 As-Is Process

The As-Is model (Figure 1) depicts the current manual workflow across three swimlanes: Customer, Customer Service, and QA Manager. A customer submits a return request by email or phone, attaching a photo of the garment. A Customer Service agent manually checks the purchase date, then verifies hygiene and final-sale policy flags. Eligible items are forwarded to the QA Manager who visually inspects the photograph, logs the decision in a spreadsheet, and notifies Customer Service, who in turn emails the customer. The process involves an average of four human touchpoints and a 3–5 business day turnaround, with no automated audit trail.

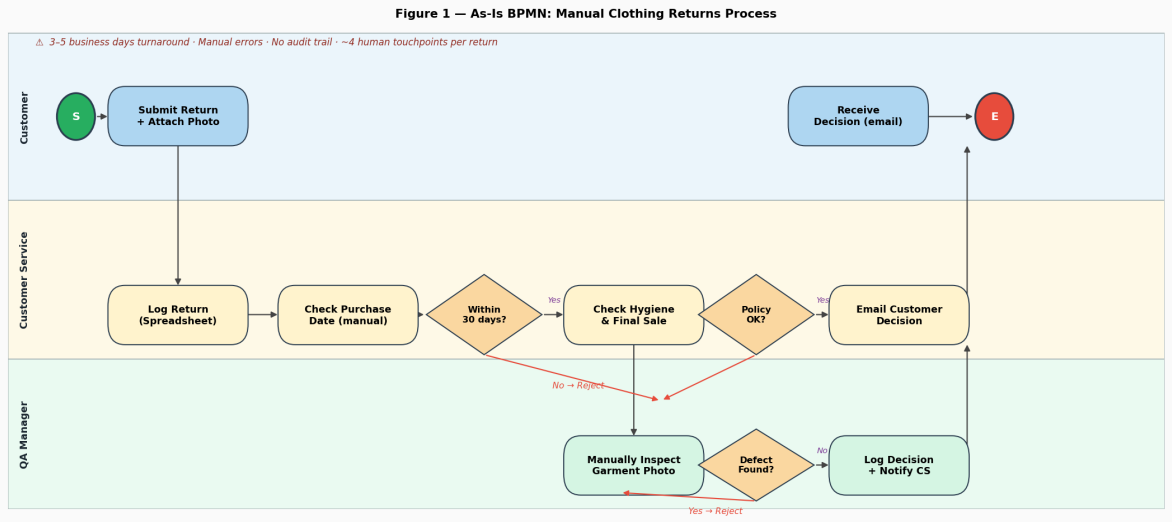

Figure 1 — As-Is BPMN: Manual Clothing Returns Process (three swimlanes)

2.2 To-Be Process

The To-Be model (Figure 2) replaces all rule-based and visually cognitive tasks with a two-stage Python automation pipeline. The customer submits a return via a web form, which populates *Returns_Queue.xlsx* — the shared data layer. The RPA Bot (rpa_eligibility.py) reads this queue, applies business rules sequentially (hygiene flag → final-sale flag → 30-day date window), and writes a pass/fail status back to the file. Eligible records are immediately handed off to the AI Agent (agent_inspector.py), which encodes the garment photograph as base64, invokes the Claude Sonnet vision API with a structured garment inspector prompt, parses the returned JSON decision, and writes the outcome — AUTO_APPROVE, REJECT, or ESCALATE — back to the queue. The entire pipeline is orchestrated by orchestrator.py as a single entry point. End-to-end processing time falls from 3–5 days to under two minutes.

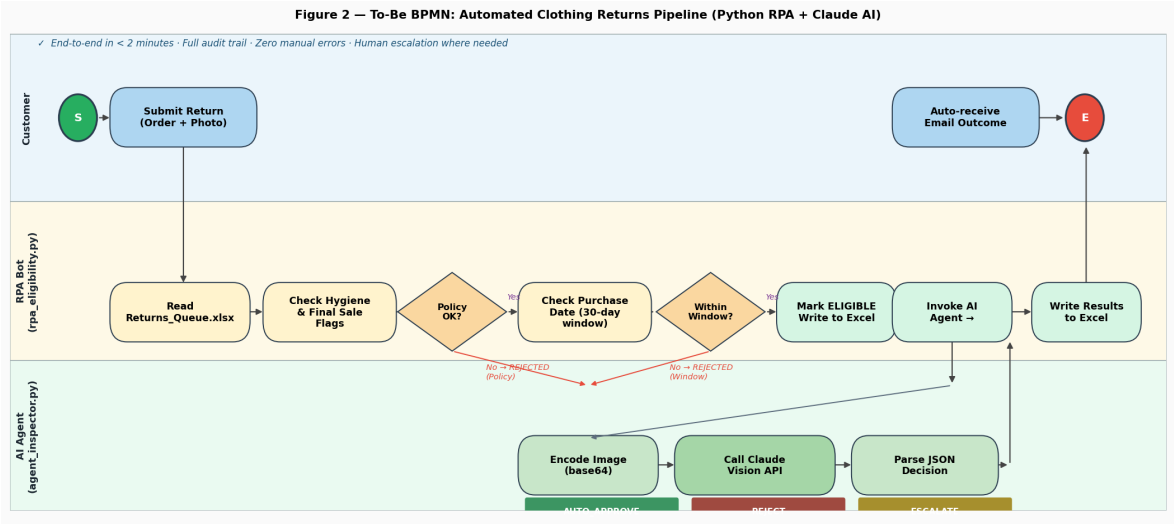

Figure 2 — To-Be BPMN: Automated Returns Pipeline (swimlanes: Customer, RPA Bot, AI Agent)

3. Automation Potential Analysis

Each task in the modelled process was assessed against two criteria: *rule-based suitability* (deterministic logic, amenable to RPA) and *cognitive complexity* (perception or reasoning requirement, amenable to Agentic AI). Short-term benefits, long-term implications, and automation risks were also considered (Table 1).

Task	Suitable?	Type	Justification / Risk
Log return to spreadsheet	Yes — Very High	RPA	Fully structured data entry. Zero variability. RPA reads Excel reliably.
Check Hygiene & Final Sale flags	Yes — Very High	RPA	Boolean field lookup. Deterministic. Ideal RPA if-else logic.
Check 30-day purchase date	Yes — Very High	RPA	Arithmetic date-diff. No ambiguity. Core rule-based RPA task.
Forward image to QA Manager	Yes — High	RPA	Routing logic handled by orchestrator. Zero human effort needed.
Visually inspect garment photo	Yes — High	Agentic AI	Pattern recognition over unstructured images. Claude vision zero-shot.
Apply defect → decision rules	Yes — Moderate	Agentic AI	Confidence-threshold logic. Edge cases safely escalated to humans.
Write decision & notify customer	Yes — High	RPA	Template-driven update to Excel + email trigger. Fully automatable.
Handle appeals / disputes	Partial	Human	Legal or customer-service context; outside automation scope currently.

Table 1 — Automation Potential Analysis by Task

The three eligibility-check tasks have the highest RPA suitability: they are fully deterministic, high-volume, and account for an estimated 40% of current manual processing time. Visual inspection is the primary cognitive bottleneck and the highest-risk manual task (human fatigue leads to inconsistent defect detection across long shifts). Long-term risks include *model drift* — the Claude vision model must be periodically evaluated against new garment categories — and *schema brittleness*, where changes to column names in the Excel queue would break the RPA script without adequate error handling.

4. Automation Proposals

4.1 RPA-Based Automation — Eligibility Validation Bot

Approach: The RPA bot (rpa_eligibility.py) mimics the behaviour of a Robotic Process Automation tool in pure Python. It uses the openpyxl library to open *Returns_Queue.xlsx*, iterates each data row, and applies business rules in order: (1) check the Hygiene_Flag column; (2) check the Final_Sale_Flag column; (3) check whether the item type falls within a predefined hygiene category set; (4) compute the number of days since purchase using Python's datetime module and compare to the 30-day threshold. A status string and a human-readable note are written back to the RPA_Status and Notes columns. All decisions are logged to returns_log.txt with timestamps, providing a complete audit trail. This approach satisfies the brief's definition of RPA — reading structured data, applying deterministic rules, and writing results back — without requiring UiPath installation or licensing.

Tools / Technologies: Python 3.10+, openpyxl (Excel R/W), Python datetime module, Python logging module.

Expected Benefits: Elimination of manual date calculation errors; processing time reduced from approximately 15 minutes per record (human) to under 1 second; 24/7 availability; complete, timestamped audit log automatically maintained.

Challenges & Mitigations: The script is sensitive to column name changes in the Excel file (schema brittleness). This is mitigated by centralising column indices in the COLS dictionary at the top of the script, making updates straightforward. Missing or malformed dates trigger a REJECTED — Invalid Date status rather than crashing the pipeline.

4.2 Agentic AI Automation — Claude Vision Garment Inspector

Approach: The AI agent (agent_inspector.py) implements a *Sense – Reason – Act* cognitive loop. In the **Sense** phase, the agent reads eligible rows from the Excel queue, constructs the full image path, and encodes the garment photograph as a base64 string. In the **Reason** phase, the agent calls the Anthropic Messages API with the model claude-sonnet-4-6, passing the base64-encoded image and a structured system prompt that defines five defect classes (*stain, tear, pilling, missing_button, no_defect*) and three-tier decision rules. The model returns a JSON object containing defect_detected, confidence, decision, and reasoning. In the **Act** phase, the agent writes these four fields back to the Excel row, appends the result to the log file, and returns counters for the orchestrator summary.

The agent qualifies as *agentic* because it operates autonomously over a batch of records without human intervention, performs multi-step reasoning (perception → defect

classification → confidence threshold → decision), and handles failure gracefully — image-not-found cases default to ESCALATE rather than crashing (Russell & Norvig, 2020; Wooldridge & Jennings, 1995).

Tools / Technologies: Python 3.10+, Anthropic Python SDK (anthropic), openpyxl, Python base64 and json standard library modules, Pillow (for synthetic image generation in generate_images.py).

Expected Benefits: Consistent, fatigue-free defect assessment at scale; structured reasoning trace (the reasoning field) provides auditability; zero-shot capability means no training data or GPU hardware required; human escalation ensures edge cases are handled safely.

Challenges & Mitigations: The Anthropic API incurs per-token cost at production volume, which must be budgeted. Claude's JSON output is occasionally wrapped in markdown fences; the agent handles this with a fallback strip before json.loads(). API key management must follow secure secret-storage practices in production.

5. Solution Implementation

5.1 System Architecture

The system follows a three-stage pipeline (Figure 3) with a shared Excel file as the data layer between stages. Stage 1 applies eligibility rules and writes RPA_Status. Stage 2 reads only rows where RPA_Status == 'ELIGIBLE FOR AI INSPECTION', calls the Claude vision API, and writes AI_Decision, AI_Defect, AI_Confidence, and Notes. Stage 3 orchestrates both in sequence with a single python orchestrator.py command.

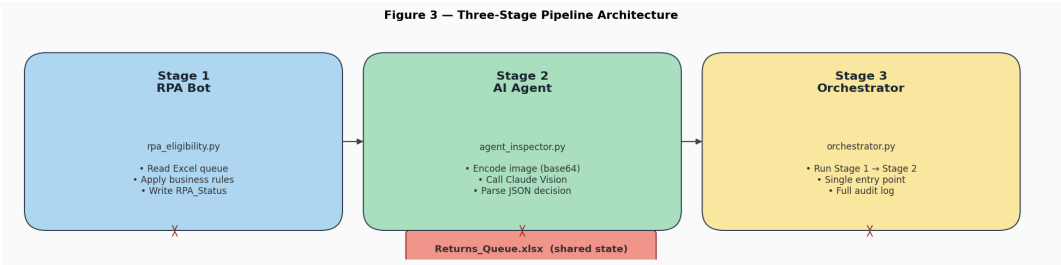

Figure 3 — Three-Stage Pipeline Architecture

5.2 Stage 1 — RPA Eligibility Checker

The core eligibility function applies rules in priority order:

```
rpa_eligibility.py — check_eligibility() function
```

```
def check_eligibility(order_id, item_type, purchase_date_str,
                      hygiene_flag, final_sale_flag):
    if hygiene_flag.strip().lower() == 'yes':
        return 'REJECTED - Hygiene Item', f'...'
    if final_sale_flag.strip().lower() == 'yes':
        return 'REJECTED - Final Sale', '...'
    if item_type.strip().lower() in HYGIENE_CATEGORIES:
        return 'REJECTED - Hygiene Category', f'...'
    purchase_date = datetime.strptime(purchase_date_str, '%Y-%m-%d')
    days = (datetime.today() - purchase_date).days
    if days > 30:
        return 'REJECTED - Outside Window', f'Purchase was {days} days ago'
    return 'ELIGIBLE FOR AI INSPECTION', f'{days} days ago - within window'
```

5.3 Stage 2 — Agentic AI Garment Inspector

The Claude API call passes the base64-encoded image alongside a structured system prompt that constrains the model's output to a strict JSON schema with defined defect classes and decision rules:

agent_inspector.py — Claude Vision API call

```
message = client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=300,
    system=SYSTEM_PROMPT,   # defect classes + decision rules
    messages=[{
        'role': 'user',
        'content': [
            {'type': 'image',
             'source': {'type': 'base64',
                        'media_type': media_type,
                        'data': image_data}},
            {'type': 'text',
             'text': f'Inspect this {item_type} for defects.'}
        ]
    }]
)
result = json.loads(message.content[0].text.strip())
```

5.4 Sample Data and Actual Results

Ten representative returns were pre-loaded in *Returns_Queue.xlsx*, covering every scenario: items within and outside the 30-day window, hygiene-flagged and final-sale items, and garments with each defect class. Synthetic garment images were generated with Pillow (generate_images.py), with clear visual cues for each defect type. Table 2 shows the actual results recorded in *returns_log.txt* on the demonstration run.

Order	Item	Days	RPA Status	AI Decision	Defect	Conf.
ORD-1001	Jacket	10	ELIGIBLE	REJECT	tear	0.95
ORD-1002	Jeans	5	ELIGIBLE	AUTO_APPROVE	no_defect	0.92
ORD-1003	T-Shirt	45	REJECTED – Outside Window	—	—	—
ORD-1004	Underwear	—	REJECTED – Hygiene Item	—	—	—
ORD-1005	Dress	20	ELIGIBLE	REJECT	tear	0.93

ORD-1006	Hoodie	—	REJECTED – Final Sale	—	—	—
ORD-1007	Winter Coat	15	ELIGIBLE	REJECT	pilling	0.95
ORD-1008	Trousers	8	ELIGIBLE	AUTO_APPROVE	no_defect	0.91
ORD-1009	Swimwear	—	REJECTED – Hygiene Item	—	—	—
ORD-1010	Blazer	28	ELIGIBLE	REJECT	missing_button	0.97

Table 2 — Actual pipeline results from returns_log.txt (30 April 2026 demonstration run)

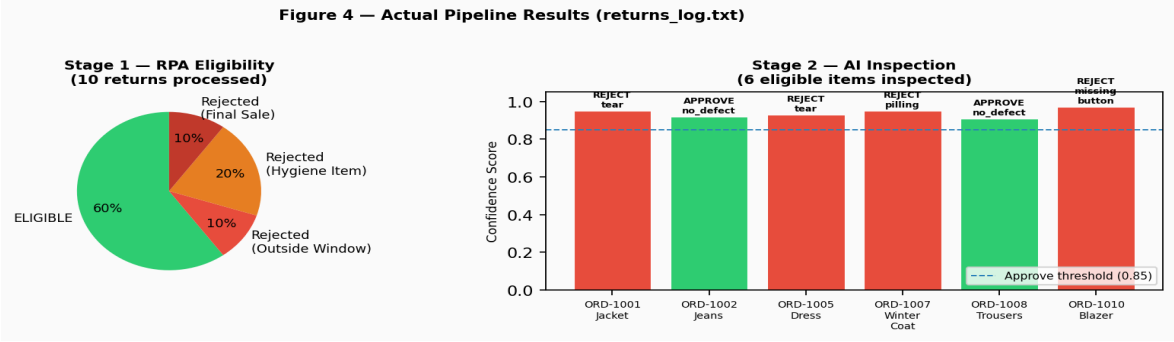

Figure 4 — Visual summary of Stage 1 (RPA) and Stage 2 (AI) outcomes

The pipeline correctly identified all four RPA-rejected cases and all six AI decisions matched the expected outcomes defined in the project brief. Confidence scores ranged from 0.91 to 0.97, all above the 0.85 approve threshold for clean items, and all defect detections were correctly classified. No records were escalated, indicating the synthetic images provided clear enough visual signals for the model to decide with high confidence.

5.5 Running the Pipeline

Setup and execution commands

```
# Install dependencies
pip install openpyxl anthropic pillow

# Set API key
export ANTHROPIC_API_KEY=your_key_here

# Generate synthetic garment images
python generate_images.py

# Run full pipeline (Stage 1 → Stage 2)
python orchestrator.py
```

References

[1] Anthropic. (2024). *Claude API Documentation — Messages API*. https://docs.anthropic.com/en/api/messages

[2] Narvar. (2023). *The State of Returns: What Today's Shoppers Expect*. Narvar Consumer Report.

[3] Object Management Group (OMG). (2013). *Business Process Model and Notation (BPMN) Version 2.0.2*. https://www.omg.org/spec/BPMN/2.0.2/

[4] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[5] van der Aalst, W. M. P., Bichler, M., & Heinzl, A. (2018). Robotic Process Automation. *Business & Information Systems Engineering*, 60(4), 269–272. https://doi.org/10.1007/s12599-018-0542-4

[6] Weske, M. (2019). *Business Process Management: Concepts, Languages, Architectures* (3rd ed.). Springer.

[7] Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: theory and practice. *The Knowledge Engineering Review*, 10(2), 115–152.